Assignmnent 4

Table of Contents

- Assignmnent 4
  - Table of Contents
  - Question 1
    - a)
    - b)
    - c)
      - i) Unlink:
      - ii) Splice:
    - d)
    - e)
    - f)
  - Question 2
    - a)
    - b)
    - c)
    - d)
    - e)
    - f)
    - g)
    - h)
  - Quesstion 3
    - a)
    - b)
    - c)
    - d)
    - e)
    - f)
    - g)
    - h)
  - Question 4
    - a)

- b)
- c)
- d)
- e)
- f)
- g)
- h)
- i)
- Question 5
  - a)
  - b)
  - c)
  - d)
  - e)
  - f)
  - g)
  - h)
  - i)
  - j)
  - k)

Question 1

a)

Assuming that the palindrome can start from any point of the linked list due to its circular nature:
The most obvious and naive approach to the problem would be the $O(N^2)$ one, where you keep two pointers - one is start and other is end which is start $\rightarrow$ left. Now start iterating `p = p->right` and `q = q->left` where p and q are initialised with start and end respectively and check the condition for palindrome. If the condition is violated, we move start to the next element and repeat this process.

b)

Algorithm

Iterate over the whole linked list once and find the index of the first element that is greater than x, if it exists and if it doesn't set idx = -1.

- Now if idx = 1, create a node z.
- Set $p = head \to left, head \to left = z, z \to left = p$ and $p \to right = z$ and $head = z$.
- For any other index, go to that respective node say y. Set $y = y \to left$, $q = y \to right$. $y \to right = z, z \to right = q$ and $q \to left = z$.
- If idx = -1, then set y=last node, $q = y \to right$. $y \to right = z, z \to right = q$ and $q \to left = z$. If Tail exists, then set Tail to z.

  Time Complexity

  As obvious, the whole algorithm is executed in an $O(N)$ Time complexity

c)

i) Unlink:

Go to the ith index node say x. Set $p = x \to left, q = x \to right, p \to right = q, q \to left = p$. If x was the first element, set head = q. If x was the last element, set tail = p. Time complexity is $O(N)$ to find the node, but $O(1)$ to unlink if the pointer is given.

ii) Splice:

For the 2nd LL referred to as CDL', set $head' \to left = NULL$ and $tail' \to right = NULL$. Now in the given linked list, go to the ith node say X. Set $X \to right \to left = tail'$ and $X \to right = head'$. Time complexity is $O(N)$ to traverse to the node, $O(1)$ for pointer manipulation.

d)

Algorithm

1. Set $k = k \pmod n$.
2. Now mark cnt = 1 and start iterating from $q = tail$ and move ahead by $q = q \to left$, at each step increment cnt by 1.
3. When $cnt = k$, break out of the loop.
4. Now set head = q and tail = q->left.

I am incrementing cnt at the end of the loop block while I check the condition at the start. Thus, I effectively move only $k-1$ steps to the left from the tail. Time complexity is $O(K)$.

e)

CPP

```cpp
vector<int> v(n+1);
v[0] = 0;
v[1] = head->w;
for(p = head->right; p != head; p = p->right) {
    v[i] = v[i-1] + p->w;
    i++;
}
vector<int> cost(n+1);
for(int i = 1; i <= n; i++) {
    cost[i] = min(v[i], v[1] + v[n] - v[i-1]);
}
```

Here I have not shown the creation of the linked list so you may take it to be the general way. The time complexity is a simple $O(N)$.

f)

This is an $O(1)$ time complexity problem. Assuming that CDL1 comes first in CDL3, simply set:

- $tail1 \rightarrow right = head2$
- $head1 \rightarrow left = tail2$
- $head2 \rightarrow left = tail1$
- $tail2 \rightarrow right = head1$
- $head = head1$
- $tail = tail2$

Question 2

a)

CPP

```
int mx1 = max(head->data, head->right->data);
int mx2 = min(head->data, head->right->data);

for (p = head->right->right; p != head; p = p->right) {
    if(p->data >= mx1) {
        mx2 = mx1;
        mx1 = p->data;
    } else if (p->data > mx2) {
        mx2 = p->data;
    }
}
cout << mx1 * mx2 << endl;
```

As clear from the code, the time complexity is a simple $O(N)$

b)

To detect a loop in a singly linked list, we can use two pointers - slow and fast. The slow pointer traverses the list one by one while the fast pointer traverses the list by moving two nodes ahead. Also, after each increment, check if nodes are equal or not. If at any point, any of the pointer = NULL, then stop iterating.
To find the first element of the loop: Keep one pointer say A at the point where intersection occured, keep the other pointer say B at start. Now start iterating, A and B will meet at the start of the loop. Why will they meet at the start of the loop? This can be seen easily using mathematics, suppose the loop length is k, the length outside the loop is x and they meet at yth position inside the loop initially, then $2(x+y) = x+y+m \cdot k$, so $x+y = mk$. This is essentially what the Floyd's tortoise and hare algorithm does. The time complexity is a simple $O(N)$.

c)

Here in this question, we can use an algorithm similar to the hash function.
Declare a vector of size $n+1$, say v, store the values corresponding to the nodes. Now start traversing through the linked list, generate a random number using `rand()` and take its mod with n in res1 i.e., $res1 = rand() \pmod{n}$.
If $res1 == 0$, increment it by n. Do this same process again for another variable say res2. Now swap `v[res1]` and `v[res2]`. Now iterate over the linked list and assign the correspondingly shuffled values from v to the nodes. The time complexity is a simple $O(N)$.

d)

Algorithm

- Create two pointers head1 and head2.
- Set head1 to the first element of type A.
- Set head2 to the first element of type B.
- Now traverse the linked list and build two separate linked lists of Type A and Type B.
- Set $tail1 \rightarrow next = head2$.
- Set $head = head1$.
- Set $tail = tail2$.

This maintains the order of elements as we iterate in order only once. This is an in place mechanism. We dont create new nodes. This is an $O(N)$ algorithm.

e)

Serialization $S(DL)$
Converts the doubly linked list into a storable format (string/array). Traverse the list from head to tail. Store node values sequentially.
Deserialization $D(S)$
Reconstructs the doubly linked list from the stored format. Create nodes one by one in the same order. Set next and prev pointers while linking nodes.
Storing in a vector would be pretty convenient in comparison to a string. The space complexity for storage in either case would be an $O(N)$ and the time complexity would also be an $O(N)$.

f)

CPP

```cpp
int sum = 0;
for (p = head->next; p != head; p = p->next) {
    sum += p->data;
    p->data = sum;
}
```

The time complexity is a simple $O(N)$.

g)

KEY IDEA: Traverse the list in segments of size k. Reverse each segment. Connect the segments properly.
For each block of size k: You reverse pointers locally. Then stitch: previous block $\rightarrow$ current block, current block tail $\rightarrow$ next block.
What we will do is that we will keep a track of the last element of previous block and first element of the next block(if any), then we will isolate the chunk i.e. unstitch it, reverse it and then stitch it back. The time complexity would be an $O(N)$ and since we dont use any additional linked list and only a few temporary variables, the space complexity is a simple $O(1)$.

h)

Given that the two linked lists intersect at one point, so the idea is that a node can be reached by various ways however once that node is reached, there is only one way out.
Assuming that m is greater than n, therefore traverse $m - n$ nodes ahead in the larger linked list. Now declare two pointers and start moving ahead in both linked lists and check for equality. If the condition gets satisfied before null pointer is encountered, we get the required position of the intersection or else there is no intersection. The time complexity would be an $O(M)$.

Quesstion 3

a)

This question is identical to 2 (D). There, we just weren't given the condition for choosing Type A and Type B. Here we have been provided with the condition in the form of a pivot element. Time complexity is $O(N)$.

b)

To check whether a palindrome could be created, we could either follow the naive $O(N^2)$ approach as in 1(A) or use a more cleaner approach that also takes $O(N^2)$ using the properites of string.

- Traverse through the string until \n is encountered. Meanwhile keep creating new nodes and assign each character to $node \rightarrow data$ and link it with others while updating the pointers.

Alternatively, we can track character frequencies. If more than one character has an odd frequency, a palindrome cannot be formed. Time Complexity: $O(N)$.

c)

To find the maximal sum path from head to tail of two intersecting/concatenating lists, we calculate the sum of elements before the intersection point for both lists, and then add the maximum of these two sums to the sum of the common path. Time complexity is $O(M + N)$.

d)

The algorithm for the given problem doesnt depend on if the linked list is doubly linked or singly linked. It is solved using the same FLOYD'S TORTOISE AND HARE PROBLEM IN 2(B). The time complexity is $O(N)$.

e)

In a balanced BST: Root = middle element, Left subtree = left half, Right subtree = right half. This is different from building from an array as Linked lists dont support random position access.

- Recursively build left subtree using first $n/2$ nodes.
- Current node becomes root. Move pointer forward.
- Recursively build right subtree.

Time Complexity is $O(N)$. Space Complexity is $O(log\ n)$ for the recursion stack (balanced tree). DLL in contrast to SLL enables sequential access, avoiding repeated traversal.

f)

CPP

```
void SR(Node*& head, int i, int j) {
    Node* p = NULL;
    Node* q = NULL;
    Node* curr = head;
    for (int cnt = 1; curr && cnt <= j; cnt++, curr = curr->next) {
        if (cnt == i) p = curr->prev;
        if (cnt == j) q = curr->next;
    }

    Node* lochead = (p) ? p->next : head;
    Node* loctail = (q) ? q->prev : NULL;

    // cut the segment
    if (loctail) loctail->next = NULL;
    if (lochead) lochead->prev = NULL;

    // reverse the segment
    curr = lochead;
    while (curr) {
        Node* temp = curr->next;
        curr->next = curr->prev;
        curr->prev = temp;
        curr = temp;
    }

    // reconnect
    Node* newHead = loctail;
    Node* newTail = lochead;
    if (p) { p->next = newHead; newHead->prev = p; }
    else { head = newHead; head->prev = NULL; }

    if (q) { newTail->next = q; q->prev = newTail; }
    else { newTail->next = NULL; }
}
```

Time complexity is $O(N)$.

g)

KEY IDEA: If node-val already exists in the set v, then inserting it to the set wont make any difference. Thus, if the size remains same before and after insertion, we know, we have to remove this node. The ideas in itself was the whole procedure. Once we know if a node is to be removed, we just have to

stitch together $curr \to prev$ and $curr \to next$. In my approach however, the properties of DLL dont do much benefit. It just provides us with the leverage of managing two pointers less and thus is less cumbersome. The time complexity is also an optimal $O(N)$.

h)

For extracting a sublist, we detach the sublist from the original list by manipulating the $prev$ and $next$ pointers of the boundaries. For inserting, we find the node at position k, find the tail of the sublist, and stitch the sublist into the main list. The edge cases include $i = 1$ (updating head), $j = n$ (no right boundary), and $k = 1$ (inserting at head). The time complexity is clearly $O(N)$ for traversal.

Question 4

a)

Algorithm

To design a stack S with a $min()$ operation in $O(1)$ time, we can use an auxiliary stack `minStack` alongside our main stack `mainStack`.

- **push(x):** Push x onto `mainStack`. If `minStack` is empty or $x \leq$ `minStack`.top(), push x onto `minStack`.
- **pop():** Pop from `mainStack`. If the popped element equals `minStack.top()`, pop from `minStack`.
- **min():** Return `minStack.top()`.

Both Time and Space complexity are $O(1)$ per operation, though the Space complexity is $O(N)$ overall due to the auxiliary stack.

b)

Algorithm

To check if a string of brackets is balanced:

- Traverse the string X.
- If the character is an opening bracket ((, {, [), push it onto the stack.
- If it is a closing bracket, check if the stack is empty (if so, return false). Pop the top of the stack and check if it matches the corresponding opening bracket type.
- After traversal, if the stack is empty, it's balanced.

Time Complexity: $O(N)$. Space Complexity: $O(N)$.

c)

Algorithm

To evaluate a postfix expression P:

- Initialize a stack. Iterate over P.
- If p_i is an operand, push it to the stack.
- If p_i is an operator, pop the top two elements (let's say A and B, where A is top and B is second to top).
- Evaluate $B \text{ operator } A$ and push the result back.
- The final result is the only element left in the stack.

Time Complexity is $O(N)$ as we traverse the string exactly once. Space Complexity is $O(N)$.

d)

Algorithm

To convert infix expression I to postfix:

- Create an empty string res and a stack for operators.
- Traverse I:
  - If operand, append to res.
  - If (, push to stack.
  - If), pop from stack to res until (is found. Pop (.
  - If operator, pop from stack to res while stack's top operator has greater or equal precedence. Then push current operator.
- Pop all remaining elements from stack to res.-

Time Complexity: $O(N)$.

e)

Algorithm

To implement a Stack using two queues Q_1 and Q_2 enforcing LIFO:

- **push(x):** Enqueue x to Q_2. Then, dequeue all elements from Q_1 and enqueue them to Q_2. Finally, swap the names of Q_1 and Q_2. This operation takes $O(N)$ time.
- **pop():** Dequeue from Q_1. This takes $O(1)$ time.

- To achieve $O(1)$ amortized push and $O(N)$ pop, we could instead make pop do the heavy lifting by transferring $N-1$ elements to Q_2.

f)

Algorithm

To find the Nearest Greater to Right (NGR) for array A:

- Initialize an array B of size N with -1. Initialize a stack.
- Traverse array A from right to left (index $n-1$ down to 0).
- For each element a_i:
  - While stack is not empty and stack top $\leq a_i$, pop the stack.
  - If stack is not empty, $b_i =$ stack top.
  - Push a_i onto the stack.

Time Complexity: $O(N)$. Every element is pushed and popped at most once.

g)

CPP

```
int MAH(vector<int>& H) {
    int n = H.size();
    stack<int> s;
    int max_area = 0;
    int i = 0;
    while (i < n) {
        if (s.empty() || H[s.top()] <= H[i]) {
            s.push(i++);
        } else {
            int top = s.top();
            s.pop();
            int area = H[top] * (s.empty() ? i : i - s.top() - 1);
            max_area = max(max_area, area);
        }
    }
    while (!s.empty()) {
        int top = s.top();
        s.pop();
        int area = H[top] * (s.empty() ? i : i - s.top() - 1);
        max_area = max(max_area, area);
    }
    return max_area;
}
```

We use a stack to maintain indices of increasing heights. When a smaller height is encountered, we compute the area of the rectangles that can be formed by the popped heights. Time Complexity: $O(N)$.

h)

Algorithm

The Stock Span Problem requires finding consecutive days before i where price $\leq p_i$.

- We use a stack to store the indices of the days.
- Traverse P from left to right.
- For each day i, pop elements from the stack while $P[\text{stack.top}()] \leq p_i$.
- If the stack becomes empty, the span is $i + 1$. Else, the span is $i - \text{stack.top}()$.
- Push index i onto the stack.

Time Complexity: $O(N)$.

i)

Algorithm

To simulate backspace string comparison:

- Traverse S_1. If char is not #, push to stack. If # and stack not empty, pop.
- Repeat the exact same process for S_2 using a second stack.
- Compare the two stacks element by element.

Time Complexity: $O(N + M)$ where N and M are string lengths.

Question 5

a)

Algorithm

To reverse queue Q:

- Dequeue all elements from Q and push them onto a Stack S.
- Pop all elements from S and enqueue them back into Q.
- Because a stack is LIFO and a queue is FIFO, this directly reverses the order.

Time Complexity: $O(N)$ since each element is enqueued, dequeued, pushed, and popped exactly once. Space Complexity is $O(N)$ for the auxiliary stack.

b)

CPP

```
vector<int> maxSlidingWindow(vector<int>& A, int k) {
    deque<int> dq;
    vector<int> res;
    for (int i = 0; i < A.size(); i++) {
        // Remove indices outside the current window
        if (!dq.empty() && dq.front() == i - k) {
            dq.pop_front();
        }
        // Remove elements smaller than current as they are useless
        while (!dq.empty() && A[dq.back()] <= A[i]) {
            dq.pop_back();
        }
        dq.push_back(i);
        // Add max to result once window is formed
        if (i >= k - 1) {
            res.push_back(A[dq.front()]);
        }
    }
    return res;
}
```

Using a deque ensures $O(N)$ time complexity as each element is pushed and popped from the deque at most once.

c)

Algorithm

Implementing a queue Q using two stacks S_1 and S_2:

- **enqueue(x):** Push x onto S_1. Time: $O(1)$.
- **dequeue():** If S_2 is empty, pop all elements from S_1 and push them onto S_2. Then pop from S_2. Amortized Time Complexity for dequeue is $O(1)$. The choice of having S_1 for input and S_2 for output optimizes the amortized cost compared to moving elements back and forth on every operation

d)

Algorithm

To interleave halves of an even-length queue using a stack:

1. Dequeue first half (N elements) and push to Stack.
2. Pop all from Stack and enqueue back to Queue. (Queue is now second half followed by reversed first half).
3. Dequeue first half (which is the original second half) and enqueue it back.
4. Dequeue first half (the reversed first half) and push to Stack. (Stack now has first half in correct order).
5. Interleave: While Stack is not empty, pop from Stack and enqueue to Queue, then dequeue from Queue and enqueue to Queue.

Time Complexity: $O(N)$

e)

Algorithm

For the Circular Tour problem:

- Keep a `total_surplus` (sum of $g_i - c_i$) and `current_surplus`.
- Start iterating from $i = 0$ with `start_index = 0`.
- Add $(g_i - c_i)$ to both surpluses.
- If `current_surplus < 0`, the journey from `start_index` to i failed. Set `start_index = i + 1` and reset `current_surplus = 0`.
- If `total_surplus` >= 0 at the end, return `start_index`, else return -1.

Time Complexity: $O(N)$. Space Complexity: $O(1)$.

f)

Algorithm

To find the first non-repeating character in a stream:

- Maintain a frequency array for the characters and a Queue to store the order of characters.
- For each new character s_i, increment its frequency and push it to the Queue.
- While the Queue is not empty and the frequency of the front element is > 1, pop the Queue.
- The front of the Queue is the first non-repeating character.

Time Complexity: $O(1)$ per character insertion. Space Complexity: $O(1)$ (assuming fixed alphabet size).

g)

Algorithm

To generate all permutations of Q:
Generating permutations purely with a queue is essentially breadth-first traversal of the state space. Start with an empty list in the queue. At each step, dequeue a partial permutation, and for every element not yet in the permutation, create a new partial permutation and enqueue it. Repeat until lengths reach N.
Complexity: $O(N \cdot N!)$ time and space, representing the total number of permutations.

h)

Algorithm

To implement a Stack using one Queue Q:

- **push(x):** Enqueue x. Then, get the current size of the queue (say s). For $s - 1$ times, dequeue an element and immediately enqueue it back. This moves the newly added element to the front of the queue.
- **pop():** Dequeue from Q.

Time Complexity: push is $O(N)$, pop is $O(1)$.

i)

Algorithm

To rotate Q by k positions:

- Set $k = k \pmod{n}$.
-
  - For $i = 1$ to k, dequeue the front element and immediately enqueue it to the back.

Time Complexity is $O(K)$.

j)

Algorithm

To support `getMax()` in $O(1)$ amortized time:

- Use a main Queue Q for standard operations, and a Deque D to maintain maximums.
- **enqueue(x):** Enqueue to Q. While D is not empty and $D.back() < x$, pop back of D. Push x to back of D.

- **dequeue():** Dequeue val from Q. If $val == D.front()$, pop front of D.
- **getMax():** Return $D.front()$.

k)

Algorithm

For Serialization $\mathcal{SQ} : Q \to S$:

- Iterate through the queue. Convert each integer q_i to a string and append it to S along with a delimiter (e.g., a comma ,).
- Time Complexity: $O(N)$.

For Deserialization $\mathcal{DQ} : S \to Q'$:

- Parse the string S using the delimiter. Convert each parsed substring back to an integer and enqueue it into Q'.
- Time Complexity: $O(N)$.